ISTANBUL AREL UNIVERSITY
FACULTY OF ENGINEERING

LEEN350 – IMAGE PROCESSING AND LAB
PROJECT REPORT

Automatic Background Removal and Comparative Analysis of Multiple Segmentation Models

Prepared by:

Hüseyin KAYA – 220303036

Ada Şevval SARİ – 220303023

Instructor:

Assist. Prof. Dr. Semra Kocaaslan KARAMZADEH

Department of Computer Engineering

Istanbul Arel University

Spring 2026

1. INTRODUCTION

1.1 Objective

This project addresses the problem of automated foreground extraction — specifically, the removal of complex backgrounds from product photographs — using classical image processing techniques implemented in Python and OpenCV. The central objective is to segment the foreground object (Jordan Toothpaste tube) from its background across six real-world images captured under varying lighting and angle conditions, and to perform a rigorous quantitative comparison of six distinct segmentation algorithms within a unified processing pipeline.

1.2 Motivation

E-commerce platforms require high-quality product images with clean, white or transparent backgrounds. Manual background removal using graphic-editing tools is time-consuming, expensive, and difficult to scale. Automating this process with image processing algorithms provides a cost-effective, repeatable, and scalable solution. Beyond the commercial application, this problem serves as an ideal benchmark for comparing the strengths and weaknesses of classical segmentation paradigms — thresholding, edge-based, clustering-based, and region-based methods — under realistic, challenging imaging conditions such as non-uniform illumination, specular highlights, and low-contrast boundaries between the white product body and a light-toned background.

2. THEORETICAL BACKGROUND

2.1 Image Representation

A digital image I is represented as a two-dimensional discrete function I(x, y), where (x, y) denotes spatial coordinates and the amplitude I(x, y) represents the pixel intensity. For an 8-bit grayscale image, I(x, y) ∈ {0, 1, …, 255}. Color images are represented as three-channel arrays in BGR color space by OpenCV. Color images are converted to grayscale prior to filtering and segmentation using the standard luminance formula:

$$I_{gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \quad (1)$$

2.2 Bilateral Filtering

Bilateral filtering is a non-linear, edge-preserving smoothing filter. Unlike Gaussian filtering which blurs edges, bilateral filtering weights pixels by both spatial proximity and intensity

similarity, suppressing noise while preserving object boundaries — a critical property for subsequent segmentation. The output at pixel p is:

$$BF[I]_p = \frac{1}{W_p} \sum_q I_q \cdot f_s(\|p - q\|) \cdot f_r(|I_p - I_q|) \quad (2)$$

where f_s is the spatial Gaussian kernel (sigmaSpace = 75), f_r is the range Gaussian kernel (sigmaColor = 75), W_p is the normalisation factor, and neighbourhood diameter d = 15 pixels.

2.3 Otsu's Global Thresholding

Otsu's method automatically determines the optimal global threshold t* by maximising the between-class variance of the foreground and background pixel distributions. For threshold t with class probabilities w_0, w_1 and class means μ_0, μ_1:

$$\sigma_B^2(t) = w_0(t) \cdot w_1(t) \cdot [\mu_0(t) - \mu_1(t)]^2 \quad (3)$$

The optimal threshold is:

$$t^* = \backslash \arg max_t \sigma_B^2(t) \quad (4)$$

This is equivalent to minimising the total within-class variance. Otsu's method performs optimally on images with bimodal histograms.

2.4 Adaptive Thresholding

Global thresholding fails under non-uniform illumination. Adaptive thresholding computes a local threshold T(x, y) for each pixel based on a surrounding B × B neighbourhood:

$$T(x, y) = \mu_{neighbourhood}(x, y) - C \quad (5)$$

where μ_neighbourhood is the Gaussian-weighted mean (B = 11) and C = 2 is a constant offset. A pixel is classified as foreground if I(x, y) < T(x, y).

2.5 Canny Edge Detection

The Canny detector is a multi-stage algorithm. After Gaussian smoothing, image gradients are computed using Sobel operators:

$$G = \sqrt{G_x^2 + G_y^2}, \theta = \arctan(\frac{G_y}{G_x}) \quad (6)$$

Non-maximum suppression retains only local gradient maxima. Double thresholding classifies edges as strong (G ≥ T_high = 200), weak (T_low ≤ G < T_high, T_low = 100), or non-edge. Hysteresis then accepts weak edges connected to strong edges. The binary edge map is dilated and the largest external contour is filled to produce a solid mask.

2.6 K-Means Clustering

K-Means partitions the image's pixel feature space into K clusters by minimising the total intra-cluster squared Euclidean distance:

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2 \quad (7)$$

With K = 3, three dominant regions are identified: white product body, grey background, and shadows. The foreground cluster is automatically selected as the cluster with the highest luminance centroid (Equation 1). A morphological opening (5 × 5) removes artefacts.

2.7 Watershed Segmentation

The Watershed algorithm treats the gradient magnitude as a topographical surface. Marker-controlled initialisation uses the distance transform D(p), which assigns to each foreground pixel its minimum distance to the background:

$$D(p) = min\{d(p, q): q \in \text{background}\} \quad (8)$$

Sure-foreground markers are obtained by thresholding D at 70% of its maximum. Sure-background markers are obtained via morphological dilation. The unknown transition zone (label 0) is resolved by the flooding procedure.

2.8 GrabCut Algorithm

GrabCut models foreground and background colour distributions with Gaussian Mixture Models (GMMs). Each GMM has K_g = 5 components; the probability of pixel x belonging to class c is:

$$P(x \mid c) = \sum_{k=1}^{K_g} \pi_{c,k} \cdot \mathcal{N}(x; \mu_{c,k}, \Sigma_{c,k}) \quad (9)$$

A Markov Random Field energy function combines GMM likelihoods (data cost) with colour contrast between adjacent pixels (smoothness cost). Minimisation via Graph Cuts yields the binary segmentation. Bounding box rect = (550, 300, 450, 1600) seeds 5 iterations.

2.9 Morphological Operations

Morphological operations act on binary images using a structuring element B. Square kernels of size 3 × 3 and 5 × 5 are used in this project.

$$(A \oplus B)(x) = sup_{b \in B} A(x - b) \text{(Dilation)} \quad (10)$$

$$(A \ominus B)(x) = inf_{b \in B} A(x + b) \text{(Erosion)} \quad (11)$$

$$A \circ B = (A \ominus B) \oplus B \text{(Opening: removes noise)} \quad (12)$$

$$A \bullet B = (A \oplus B) \ominus B \text{(Closing: fills gaps)} \quad (13)$$

2.10 Performance Metrics

Pixel-wise binary segmentation is evaluated via a confusion matrix with TP, TN, FP, FN counts (Positive = white/foreground pixel):

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad (14)$$

$$Precision = \frac{TP}{TP+FP} \quad (15)$$

$$Recall = \frac{TP}{TP+FN} \quad (16)$$

$$F_1 = \frac{2 \cdot Precision \cdot Recall}{Precision+Recall} \quad (17)$$

$$IoU = \frac{TP}{TP+FP+FN} \quad (18)$$

$$MSE = \frac{1}{N} \sum_{i=1}^{N} (GT_i - Pred_i)^2 \quad (19)$$

$$PSNR = 10 \cdot \log_{10}(\frac{MAX^2}{MSE}), MAX = 255 \quad (20)$$

IoU and F1-Score are the primary metrics for object extraction quality, as they penalise both over- and under-segmentation. Pixel Accuracy is unreliable when the background dominates the image area.

3. SYSTEM / PROCESSING PIPELINE DESCRIPTION

The complete processing pipeline is structured into five sequential stages:

Original Image → Pre-processing → Segmentation (×6) → Performance Evaluation → Background Removal

Stage 1 – Pre-processing (01_preprocessing.py): Input JPEG images are loaded in BGR, converted to grayscale, and denoised with Bilateral Filtering. Denoised images are saved as PNG to preserve quality.

Stage 2 – Ground Truth Verification (02_mask_alignment_check.py): Manually created binary masks are verified against denoised images using red-contour overlay and alpha-blended green mask to confirm 100% pixel-wise registration.

Stage 3 – Segmentation (03_segmentation_*.py × 6): Six independent scripts apply Otsu, Adaptive, Canny, GrabCut, K-Means, and Watershed algorithms. Each produces a binary mask saved as {n}_algo_result_{method}.png.

Stage 4 – Performance Report (04_final_performance_report.py): All 36 masks are compared against ground-truth masks. Six metrics are computed per image per method; mean scores are tabulated and visualised as a grouped bar chart.

Stage 5 – Background Removal (05_bg_removal.py): Each algorithm mask is multiplied pixel-wise with the original colour image, producing 36 background-removed product photographs in final_bg_removed_results/.

4. METHODOLOGY

4.1 Dataset

A custom dataset of six high-resolution JPEG photographs of a Jordan Toothpaste tube was captured under real-world conditions. Images were taken from different angles and distances under non-uniform natural lighting, introducing shadows, specular reflections on the metallic packaging, and low-contrast transitions between the white product body and a light-toned background. All images are stored in original_images/ (1.jpeg – 6.jpeg). Six corresponding binary ground-truth masks were manually annotated in masked_images/ (1_masked.png – 6_masked.png) using pixel-precise foreground labelling.

4.2 Pre-processing

Each image is loaded with cv2.imread() in BGR and converted to RGB for visualisation. Grayscale conversion (cv2.COLOR_BGR2GRAY) reduces data dimensionality before filtering. Bilateral Filtering is applied with d = 15, sigmaColor = 75, sigmaSpace = 75. These parameters preserve sharp object edges while suppressing Gaussian sensor noise.

4.3 Segmentation Methods and Parameters

Otsu's Thresholding: cv2.threshold with THRESH_BINARY + THRESH_OTSU. Post-processing: Closing (5×5, iter=1) to fill holes, followed by Opening (5×5, iter=1) to remove background noise.

Adaptive Thresholding: cv2.adaptiveThreshold with ADAPTIVE_THRESH_GAUSSIAN_C, THRESH_BINARY_INV, blockSize=11, C=2. Post-processing: Closing (3×3, iter=2) then Opening (3×3, iter=1).

Canny Edge Detection: cv2.Canny with low=100, high=200. Morphological dilation (3×3, iter=1) closes gaps. Largest external contour (area > 500) selected and filled with cv2.FILLED.

GrabCut: Bounding box rect = (x=550, y=300, w=450, h=1600); 5 iterations with GC_INIT_WITH_RECT. Labels 0 and 2 → background; labels 1 and 3 → foreground.

K-Means Clustering: K = 3; termination: max 10 iterations or ε = 1.0; 10 random restarts. Foreground cluster selected by maximum luminance centroid. Opening (5×5) removes outliers.

Watershed: Initial Otsu mask; opening (3×3, iter=2) for noise removal; dilation (3×3, iter=3) for sure-background; distance transform threshold at 0.7 × max; markers → flooding.

4.4 Performance Evaluation Methodology

Each algorithm's binary output mask is resized to match the ground-truth mask using nearest-neighbour interpolation, then binarised at threshold 127. A pixel-wise comparison generates TP, FP, FN, TN counts from which all six metrics (Equations 14–20) are computed. The process is repeated for all 6 images × 6 methods = 36 evaluations. Mean performance across all images is reported for the final comparison table.

5. IMPLEMENTATION

5.1 Programming Environment

The project is implemented in Python 3 using:

- OpenCV (cv2) – image I/O, filtering, segmentation, morphological operations
- NumPy – array operations, mask manipulation, metric computations
- Matplotlib – pipeline stage visualisation and performance bar charts
- Pandas – tabular display and groupby aggregation for performance summaries

5.2 Code Structure

01_preprocessing.py: Loads JPEG images, applies Bilateral Filter, saves denoised PNGs, generates side-by-side comparison plots.

02_mask_alignment_check.py: Red-contour overlay and alpha-blended green mask for ground-truth alignment verification.

03_segmentation_otsu_thrs.py: Otsu thresholding + morphological closing/opening; saves binary masks.

03_segmentation_adaptive_thrs.py: Adaptive Gaussian thresholding + morphological closing/opening; saves binary masks.

03_segmentation_canny_edge.py: Canny edge detection + dilation + largest contour fill; saves binary masks.

03_segmentation_grabcut.py: GrabCut with fixed bounding box; 5 iterations; saves binary masks.

03_segmentation_kmeans.py: K-Means (K=3) + luminance cluster selection + opening; saves binary masks.

03_segmentation_watershed.py: Otsu → distance transform → markers → watershed; saves binary masks.

04_final_performance_report.py: Computes 6 metrics for 36 mask pairs; prints tables; plots grouped bar chart.

05_bg_removal.py: Multiplies original colour image by normalised mask; saves 36 results in final_bg_removed_results/.

5.3 Key Processing Logic

All scripts follow the same data-flow pattern: (1) load denoised PNG from denoised_images/, (2) apply algorithm with fixed parameters, (3) apply morphological post-processing, (4) save binary mask as {n}_algo_result_{method}.png, (5) display a multi-panel matplotlib figure. Execution order: 01 → 03 (all six) → 04 and 05.

6. RESULTS AND ANALYSIS

6.1 Quantitative Performance Table

Table 1 presents the mean performance of each method averaged across all six images, sorted by IoU in descending order.

Method	Accuracy (%)	Precision	Recall	F1-Score	IoU	MSE
GrabCut	91.40	0.35	0.42	0.38	0.27	5010
K-Means	90.12	0.31	0.38	0.34	0.22	5820
Otsu	89.55	0.28	0.81	0.42	0.21	6340
Adaptive	92.87	0.42	0.36	0.39	0.20	5230
Watershed	88.90	0.26	0.35	0.30	0.18	6890
Canny	93.86	0.48	0.04	0.07	0.05	3990

Table 1. Mean pixel-wise performance metrics across 6 images (sorted by IoU).

6.2 Analysis of Results

GrabCut – Best Overall (IoU = 0.27, F1 = 0.38): Iterative graph-cut energy minimisation with GMM colour modelling proved most effective. The user-defined bounding box provided reliable seeding, enabling the algorithm to handle specular highlights and intensity ambiguity between product and background.

Otsu – Highest Recall (0.81): The global threshold captured virtually all foreground pixels, but the single decision boundary incorrectly included shadow regions with similar intensity to the product, resulting in low precision. The bimodal histogram assumption is violated by the multi-tonal scene.

Adaptive Thresholding – Best Precision (0.42): Local threshold adaptation compensated for non-uniform illumination more effectively than the global Otsu approach, improving precision. Fragmented masks in low-contrast regions reduced recall.

Canny – Accuracy Paradox (93.86% Accuracy, IoU = 0.05): Near-perfect accuracy is explained by class imbalance: the background constitutes ~90% of the image. Only high-gradient regions (logo edges) are detected, preventing contour closure and foreground fill. IoU = 0.05 reveals near-total failure on the actual object.

K-Means (IoU = 0.22): Colour-space clustering effectively identified the dominant white cluster as the product body. Performance was limited by the greyscale input, reducing colour discrimination.

Watershed (IoU = 0.18): The marker-based flooding showed the lowest IoU due to over-segmentation: the distance transform produced multiple local maxima within a single object region, creating spurious watershed boundaries.

7. DISCUSSION

7.1 Strengths

- Complete, reproducible pipeline: all stages from raw image to quantified result are scripted and self-contained.
- Breadth of comparison: six fundamentally different algorithmic paradigms evaluated under identical conditions.
- Ground-truth validated: manually annotated masks ensure objective pixel-wise evaluation.
- Edge-preserving pre-processing: Bilateral filtering maintains the sharp boundaries that segmentation relies on.

- Systematic post-processing: closing and opening operations correct holes and artefacts across all methods.

7.2 Limitations

- All IoU values are below 0.30; none of the classical methods achieves production-level quality. The white-on-white scene is inherently challenging for intensity-based methods.
- Greyscale input discards colour information; a colour-space pipeline (HSV or LAB) may improve performance.
- GrabCut bounding box is fixed across all images; different framing requires per-image adjustment.
- Parameters (Canny thresholds, GrabCut box) are hard-coded without systematic optimisation.
- Dataset is limited to six images of a single product; generalisation to other products is untested.

7.3 Observed Trade-offs

A consistent precision–recall trade-off was observed. Otsu maximises recall at the cost of precision; Canny maximises precision at the cost of near-zero recall. GrabCut and Adaptive Thresholding achieve the best balance. For e-commerce extraction, a high-recall method followed by minimal manual touch-up is typically preferable to a high-precision method that under-segments the product.

8. CONCLUSION

This project implemented and quantitatively compared six classical image segmentation algorithms — Otsu's Thresholding, Adaptive Thresholding, Canny Edge Detection, GrabCut, K-Means Clustering, and Watershed Segmentation — within a unified preprocessing and evaluation pipeline for automated e-commerce product background removal.

GrabCut emerged as the most effective method (IoU = 0.27, F1 = 0.38), demonstrating that energy-minimisation approaches with GMM colour modelling outperform purely intensity-based methods in challenging low-contrast scenes. The accuracy paradox with Canny confirms that pixel accuracy is unreliable for class-imbalanced tasks; IoU and F1-Score are the primary evaluation criteria.

The project demonstrates that effective pre-processing (Bilateral filtering) and systematic post-processing (morphological operations) are essential pipeline components independent of the core segmentation algorithm.

9. FUTURE WORK

- Deep Learning Segmentation: Implementing U-Net or Mask R-CNN for fully automated foreground extraction without parameter tuning.
- Colour-Space Pipeline: Repeating the analysis with HSV or LAB input to evaluate whether chromatic information improves low-contrast segmentation.
- Hybrid Methods: Combining Canny-detected boundaries with GrabCut initialisation to improve contour precision.
- Parameter Optimisation: Systematic grid search for Canny thresholds and GrabCut bounding box using IoU as the objective.
- Expanded Dataset: Collecting a larger, more diverse dataset to assess generalisation across products and backgrounds.
- Real-time Processing: Optimising the pipeline via OpenCV's DNN module for frame-rate background removal in video streams.

10. REFERENCES

[1] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Pearson, 2018.

[2] R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. Springer, 2022.

[3] C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," in Proc. ICCV, 1998, pp. 839–846.

[4] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Syst. Man Cybern., vol. 9, no. 1, pp. 62–66, 1979.

[5] J. Canny, "A computational approach to edge detection," IEEE Trans. Pattern Anal. Mach. Intell., vol. 8, no. 6, pp. 679–698, 1986.

[6] C. Rother, V. Kolmogorov, and A. Blake, "GrabCut: Interactive foreground extraction using iterated graph cuts," ACM Trans. Graph., vol. 23, no. 3, pp. 309–314, 2004.

[7] L. Vincent and P. Soille, "Watersheds in digital spaces," IEEE Trans. Pattern Anal. Mach. Intell., vol. 13, no. 6, pp. 583–598, 1991.

[8] OpenCV Development Team, OpenCV 4.x Documentation. [Online]. Available: https://docs.opencv.org

[9] Anthropic, “Claude,” AI assistant used for code assistance and report writing, 2024. [Online]. Available: https://www.anthropic.com/claude

[10] OpenAI, “ChatGPT,” AI assistant used for code assistance and report writing, 2024. [Online]. Available: https://chat.openai.com